Course Outline: Docker

Title: Docker **Skill:** Skill 34: Construir aplicaciones en contenedores **Learnpack:** + Docker **File:** s34-w12d35-docker **Prerequisite:** Contenedores (Sin hablar de Docker) **Target Audience:** Beginner developers who understand containers conceptually and are ready to use Docker in practice **Total Lessons:** 11 **Format:** Conceptual (0% hands-on)

Course Description

You already know what containers are and why they matter. Docker is the tool that makes them real. This course takes you from theory to practice — not by running code, but by building a solid mental model of how Docker works: what images and containers mean in Docker's world, how to pull and run containers from Docker Hub, and how to write a Dockerfile that packages your own application. By the end, you'll understand every step from writing a Dockerfile to having a container running on any machine.

Course Philosophy

This course emphasizes:

- Building on prior knowledge — containers are not re-explained from scratch
- Understanding before doing — the Dockerfile and Docker commands will make sense because you'll understand what they're doing
- Precision — each Docker concept (image, container, layer, registry, instruction) has a specific meaning; this course draws clear lines between them

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Docker Fundamentals	4
02 - The Dockerfile	3
03 - Assessment	1
04 - Outro	1
Total	**11**

00.0 From Theory to Tool 📖

Learning Objectives:

- Recognize Docker as the practical implementation of the container concept already learned
- Understand what this course will cover and how the sections connect
- Identify why knowing Docker specifically — not just containers in general — matters for developers

Content Outline:

- A brief bridge: what you already know (containers, isolation, reproducibility)
- What Docker adds: a concrete tool, a standard format, and an ecosystem
- Course preview: fundamentals → Docker Hub → commands → Dockerfile → build

Transitions: This lesson bridges the previous conceptual course to this practical one. The next section dives into Docker specifically — what it is and how it organizes the world of containers.

Key Principles:

- Docker is not a synonym for containers — it is the most widely used tool for creating and running them
- Docker standardized container technology in a way that made it accessible to every developer

Examples:

- Containers existed as a Linux concept before Docker; Docker made them easy enough for any developer to use
- When a job posting says "Docker experience required," it means knowing images, containers, Dockerfiles, and the Docker workflow — not just the concept of isolation

---

01.0 Docker 📖

Learning Objectives:

- Describe what Docker is and the problem it was designed to solve
- Distinguish between Docker the tool and the broader concept of containers
- Identify the core components of the Docker ecosystem

Content Outline:

- What Docker is: a platform for building, shipping, and running containers
- The problem Docker solves: packaging an application with all its dependencies so it runs identically anywhere
- Docker's ecosystem at a glance: images, containers, Dockerfile, Docker Hub, Docker Engine
- Why Docker became the standard

Transitions: This lesson introduces Docker as a whole. The next three lessons zoom into each core piece: images and containers (01.1), Docker Hub (01.2), and the commands that tie them together (01.3).

Key Principles:

- Docker packages not just your code but your entire runtime environment

- "Build once, run anywhere" is only possible because Docker standardizes the packaging format
- Docker separates the act of building an image from the act of running a container — these are distinct steps

Examples:

- A Python web app depends on Python 3.11, specific pip packages, and environment variables. Docker bundles all of this into a single image that any machine with Docker can run
- Before Docker, shipping software meant writing setup guides; with Docker, the setup IS the image

---

01.1 Images and Containers 📖

Learning Objectives:

- Define what a Docker image is and how it differs from a running container
- Explain how Docker images are built from layers
- Describe the relationship: one image → many containers

Content Outline:

- Docker images: a read-only template that defines an environment (OS, runtime, files, config)
- Layers: how images are built incrementally — each instruction in a Dockerfile adds a layer
- Why layers matter: caching, sharing, efficiency
- Containers: a running instance of an image — writable, isolated, ephemeral
- The image → container relationship: one image can spawn many containers simultaneously
- Container lifecycle: created → running → stopped → removed

Transitions: Now that you understand images and containers as Docker concepts, the next lesson introduces Docker Hub — the place where images are stored and shared.

Key Principles:

- An image is the recipe; a container is the dish — you can make many dishes from one recipe
- Images are immutable; containers are where state lives at runtime
- Layers enable Docker's efficiency: if two images share a base layer, that layer is stored once

Examples:

- The official `python:3.11` image contains a Linux OS + Python 3.11. Running it creates a container where you can execute Python code
- If you run `python:3.11` five times simultaneously, Docker uses the same image but creates five independent containers — each with its own isolated filesystem
- A change made inside a running container (like creating a file) disappears when the container is removed, unless explicitly persisted

---

01.2 Docker Hub 📖

Learning Objectives:

- Describe what Docker Hub is and what problem it solves
- Distinguish between official images, verified publisher images, and community images
- Explain what an image tag is and how to read a full image reference

Content Outline:

- Docker Hub: the public registry where Docker images are stored and shared
- What a registry is: a server that stores and serves Docker images
- Official images: maintained by Docker and language/tool owners (e.g., `python`, `node`, `postgres`)
- Verified publisher images: maintained by trusted companies (e.g., `nginx`, `redis`)
- Community images: anyone can publish; use with caution
- Image references: `[registry/]username/image:tag` — reading the full name
- Tags: version identifiers on an image (e.g., `python:3.11`, `python:3.11-slim`, `python:latest`)
- Why `latest` is not always a good idea: reproducibility risks

Transitions: You now know where images come from. The next lesson covers the Docker commands that let you pull those images down and run them as containers.

Key Principles:

- Docker Hub is to images what npm is to packages — a public repository you pull from
- Always prefer official or verified images over unknown community ones
- Pin image tags explicitly (e.g., `python:3.11`) rather than using `latest` to ensure reproducibility

Examples:

- `docker pull python:3.11` downloads the official Python 3.11 image from Docker Hub
- `postgres:16-alpine` means: the `postgres` official image, version 16, built on Alpine Linux (a minimal, lightweight base)
- If your Dockerfile uses `FROM python:latest` today and `latest` becomes 3.13 tomorrow, your build behavior changes silently — this is why pinned tags matter

---

01.3 Docker Commands 📖

Learning Objectives:

- Use the most common Docker CLI commands to manage images and containers
- Read the output of `docker ps` and `docker images`
- Distinguish between commands that act on images vs. commands that act on containers

Content Outline:

- The Docker CLI: how you interact with Docker from the terminal
- Image commands:
  - `docker pull <image>` — download an image from a registry
  - `docker images` — list locally available images
  - `docker rmi <image>` — remove a local image

- Container commands:
    - `docker run <image>` — create and start a container from an image
    - `docker run -it <image>` — run interactively (open a shell)
    - `docker run -d <image>` — run in detached (background) mode
    - `docker run -p 8080:80 <image>` — map host port to container port
    - `docker ps` — list running containers
    - `docker ps -a` — list all containers (including stopped)
    - `docker stop <container>` — stop a running container
    - `docker rm <container>` — remove a stopped container
    - `docker exec -it <container> bash` — open a shell inside a running container
- Build commands (preview): `docker build` — introduced here, covered in depth in Section 02

Transitions: You now know how to pull images and manage containers. The next section shifts focus to creating your own images — which requires writing a Dockerfile.

Key Principles:

- Every Docker command targets either an image or a container — knowing which is which prevents confusion
- `-d` and `-it` are the two most common run modes: background services use `-d`, interactive exploration uses `-it`
- Port mapping (`-p`) is how you expose a container's internal port to your host machine

Examples:

- `docker run -d -p 5432:5432 postgres:16` starts a Postgres database container in the background, accessible on your machine's port 5432
- `docker ps` shows the container ID, image name, status, and port mappings for every running container
- `docker exec -it my-container bash` drops you into a shell inside a running container — useful for debugging

---

02.0 The Dockerfile 📖

Learning Objectives:

- Describe what a Dockerfile is and its role in the Docker workflow
- Explain the relationship between a Dockerfile, a Docker image, and a running container
- Identify what information belongs in a Dockerfile

Content Outline:

- What a Dockerfile is: a plain text file with instructions for building a Docker image
- Where it fits in the workflow: Dockerfile → `docker build` → image → `docker run` → container
- What a Dockerfile defines: the base OS, the files to copy, the dependencies to install, the command to run
- Why Dockerfiles exist: to make image creation reproducible, version-controlled, and shareable
- A Dockerfile is code: it lives in your repo alongside your application

Transitions: This lesson frames what a Dockerfile is conceptually. The next lesson covers each instruction in detail, and the one after shows how to turn that Dockerfile into a running container.

Key Principles:

- A Dockerfile is the source of truth for your image — just like source code is the truth for your application
- Anyone with the Dockerfile can reproduce the exact same image
- A Dockerfile describes the environment, not the application logic

Examples:

- A Python API project has three files: `main.py`, `requirements.txt`, and `Dockerfile`. The Dockerfile tells Docker how to set up the environment and run `main.py`
- Sharing a Dockerfile is more powerful than sharing setup instructions — Docker will execute the Dockerfile exactly the same way on every machine

---

02.1 Dockerfile Instructions 📖

Learning Objectives:

- Write the five core Dockerfile instructions correctly: FROM, WORKDIR, COPY, RUN, CMD
- Explain what each instruction does and when it executes
- Distinguish between instructions that run at build time vs. instructions that run at container start

Content Outline:

- `FROM <image>:<tag>` — specifies the base image; every Dockerfile starts here
- `WORKDIR <path>` — sets the working directory inside the image for all subsequent instructions
- `COPY <src> <dest>` — copies files from the host into the image at build time
- `RUN <command>` — executes a shell command during the build (e.g., `pip install`, `apt-get install`)
- `CMD ["executable", "arg"]` — defines the default command to run when a container starts (not at build time)
- Build time vs. runtime: FROM, WORKDIR, COPY, RUN run during `docker build`; CMD runs during `docker run`
- Instruction order matters: Docker caches each layer; put instructions that change rarely near the top

Transitions: You can now write a Dockerfile. The next lesson covers building that Dockerfile into an image and running it — completing the full workflow.

Key Principles:

- `FROM` is always first — there is no Dockerfile without a base image
- `RUN` is for setup commands that modify the image; `CMD` is for the command your application actually runs
- Copy `requirements.txt` and install dependencies before copying your source code — this maximizes Docker's build cache

Examples:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

- `FROM python:3.11-slim` — use the slim Python 3.11 image as the base (smaller than the full image)
- `WORKDIR /app` — all subsequent paths are relative to `/app` inside the image
- `COPY requirements.txt .` — copy only the requirements file first (so the next RUN layer is cached when source code changes)
- `RUN pip install -r requirements.txt` — install dependencies at build time, not at runtime
- `COPY . .` — copy all remaining source files into the image
- `CMD ["python", "main.py"]` — run this when a container starts

02.2 Building Your Image 📖

Learning Objectives:

- Use `docker build` to create an image from a Dockerfile
- Tag an image correctly for identification and sharing
- Describe Docker's build cache and how it affects rebuild speed
- Apply key best practices for writing production-quality Dockerfiles

Content Outline:

- `docker build -t <name>:<tag> .` — build an image from the Dockerfile in the current directory
- The build context: what Docker sends to the engine when you run `docker build`
- Tags: naming your image (`myapp:1.0`, `myapp:latest`)
- Running your own image: `docker run myapp:1.0`
- Docker build cache: how Docker reuses layers to speed up subsequent builds
- When cache is invalidated: when a file changes or a layer above it changes
- Best practices:
  - Use a specific base image tag, not `latest`
  - Use `.dockerignore` to exclude unnecessary files from the build context (`.git`, `__pycache__`, `.env`)
  - Minimize the number of `RUN` layers by chaining commands with `&&`
  - Copy only what you need — don't `COPY . .` blindly if most files are irrelevant
  - Never put secrets (API keys, passwords) in a Dockerfile — they become part of the image

Transitions: This lesson completes the Docker workflow. The assessment is next — it covers everything from the Docker ecosystem to the Dockerfile and build process.

Key Principles:

- The build context is everything in the directory you pass to `docker build` — a large context slows builds; `.dockerignore` trims it
- Cache efficiency is a skill: ordering your Dockerfile instructions deliberately can make rebuilds nearly instant
- A Dockerfile is a security document — anything baked into the image is readable by anyone who has the image

Examples:

- `docker build -t my-python-api:1.0 .` builds the image and tags it `my-python-api:1.0`
- After changing `main.py` and rebuilding: Docker reuses the cached layers for `FROM`, `WORKDIR`, `COPY requirements.txt`, and `RUN pip install` — only the `COPY . .` layer rebuilds
- A `.dockerignore` file containing `.git`, `.env`, `__pycache__`, and `*.pyc` prevents those files from being sent to Docker and potentially baked into the image

---

03.0 What You Know Now 🧠

Learning Objectives:

- Demonstrate understanding of Docker's core concepts: images, containers, layers, Docker Hub
- Show ability to read and reason about Docker commands and their effects
- Demonstrate understanding of the Dockerfile instructions and the build workflow

Question Format: Scenario-based (single correct answer per scenario)

Questions:

1. You run `docker run python:3.11` twice simultaneously. What happens?
    - a) An error occurs because the image is already in use
    - b) Two independent containers are created from the same image ✓
    - c) The second run reuses the same container as the first
    - d) Docker downloads the image twice
2. What is the difference between a Docker image and a Docker container?
    - a) An image runs code; a container stores code
    - b) An image is a running instance; a container is a static file
    - c) An image is a read-only template; a container is a running instance of that template ✓
    - d) There is no difference — they are the same thing
3. You're writing a Dockerfile for a Python app. In what order should you write these instructions?
    - a) FROM → COPY . . → COPY requirements.txt . → RUN pip install → CMD
    - b) FROM → WORKDIR → COPY requirements.txt . → RUN pip install → COPY . . → CMD ✓
    - c) FROM → WORKDIR → RUN pip install → COPY requirements.txt . → COPY . . → CMD
    - d) FROM → CMD → WORKDIR → COPY . . → RUN pip install

4. What does `docker run -d -p 3000:80 myapp:1.0` do?

   - a) Runs myapp:1.0 interactively and exposes port 80 on the host
   - b) Runs myapp:1.0 in the background, making the container's port 80 accessible on host port 3000 ✓
   - c) Builds myapp:1.0 and runs it on port 3000 inside the container
   - d) Downloads myapp:1.0 from Docker Hub and runs it

5. You update `main.py` and rebuild your image. Docker uses the cache for all layers except one. Which layer is rebuilt?

   - a) `FROM python:3.11-slim`
   - b) `RUN pip install -r requirements.txt`
   - c) `COPY . .` ✓
   - d) `WORKDIR /app`

6. What is the purpose of a `.dockerignore` file?

   - a) It prevents Docker from running certain containers
   - b) It specifies which Dockerfile instructions to skip during build
   - c) It excludes files from the build context sent to the Docker engine ✓
   - d) It lists images that should not be pulled from Docker Hub

7. A colleague uses `FROM python:latest` in their Dockerfile. What is the risk?

   - a) Docker will refuse to build because `latest` is not a valid tag
   - b) The image will be larger than necessary
   - c) The behavior may change silently when `latest` points to a new Python version ✓
   - d) The image will be stored with no tag, making it hard to find

8. You need to install system packages (`apt-get install curl`) and Python packages (`pip install`) in your Dockerfile. What is the best approach?

   - a) Use two separate `RUN` instructions, one for each
   - b) Use one `RUN` instruction chaining both commands with `&&` to minimize layers ✓
   - c) Use `CMD` to install them at container start
   - d) Put both commands in a shell script and use `COPY` to add it

Evaluation Criteria:

- 7–8 correct: Strong understanding of Docker concepts and workflow
- 5–6 correct: Good grasp of fundamentals; review cache behavior and command flags
- Below 5: Review the Dockerfile section and Docker commands before moving on

Score Interpretation: This assessment covers the full Docker workflow from images to containers to Dockerfiles. Every question reflects a real decision a developer makes when working with Docker.

04.0 Outro

Content Outline:

- Course recap: what students accomplished
- Connection to bigger picture
- Next steps and continued learning
- Resources for further exploration
- Encouragement and celebration

Recap: You started this course already understanding containers conceptually. You've now added the practical layer: you know what Docker images and containers are in Docker's own terms, where images come from (Docker Hub), how to run and manage containers from the CLI, and how to write a Dockerfile that packages your own application. You can read a Dockerfile and understand exactly what it does — and you understand why the instruction order matters.

Connection to the Bigger Picture: Docker is the entry point to a much larger ecosystem. Dockerfiles are the foundation of everything that comes next: deployment pipelines, container orchestration (Kubernetes), cloud platforms (AWS ECS, Google Cloud Run), and CI/CD automation. Every modern backend deployment you encounter will involve Docker or something built on top of it.

Next Steps:

- Try reading real Dockerfiles on GitHub — most open-source projects include them
- When you build a FastAPI or Node project, write a Dockerfile for it
- Explore Docker Compose: a tool for running multiple containers together (a database + an API + a frontend) with a single file
- Learn about multi-stage builds: a technique for building smaller production images

Resources:

- Docker official documentation: docs.docker.com
- Docker Hub: hub.docker.com
- Play with Docker: labs.play-with-docker.com — a free browser-based Docker environment

Note: This is conclusion only — no theory, exercises, or assessment.
